

Single Entity-Ownership Model — Snake Engine

Branch: `game-with-engine-architecture` · **Date:** 2026-07-08 · **Scope:** entity / component lifetime across `src/engine` , `src/snake-game` , `src/renderer-2d`

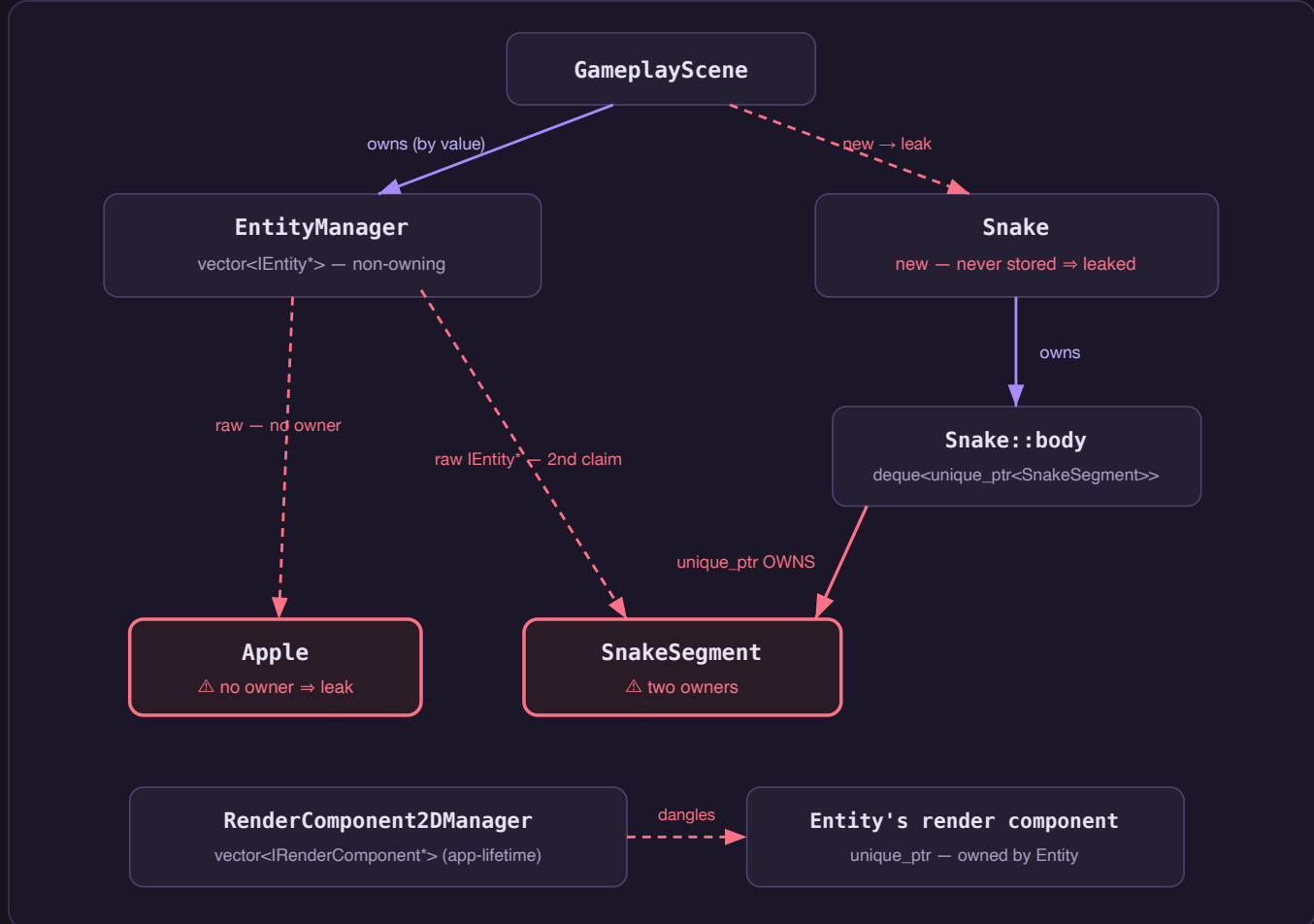
1. Executive summary

The reported double-free (F1/F2) and the live null-deref crash (F8) are three faces of one root cause: **the same objects are reachable through owners with incompatible lifetime assumptions**. Entities are `new` ed and handed to a raw-pointer container while their sub-objects are held by `unique_ptr` elsewhere; some objects have two owners, others have none. This document proposes a **single-owner model** — exactly one `unique_ptr` owner per object, everyone else a non-owning observer — and shows how it dissolves F1, F2, F3, F4, F8, F9 and F13 together.

The rule: *Exactly one owner (`unique_ptr`) per object. Everyone else holds a non-owning observer (`T*` / `T&`) and must de-register before the owner's destructor runs.*

2. Where we are today — tangled ownership

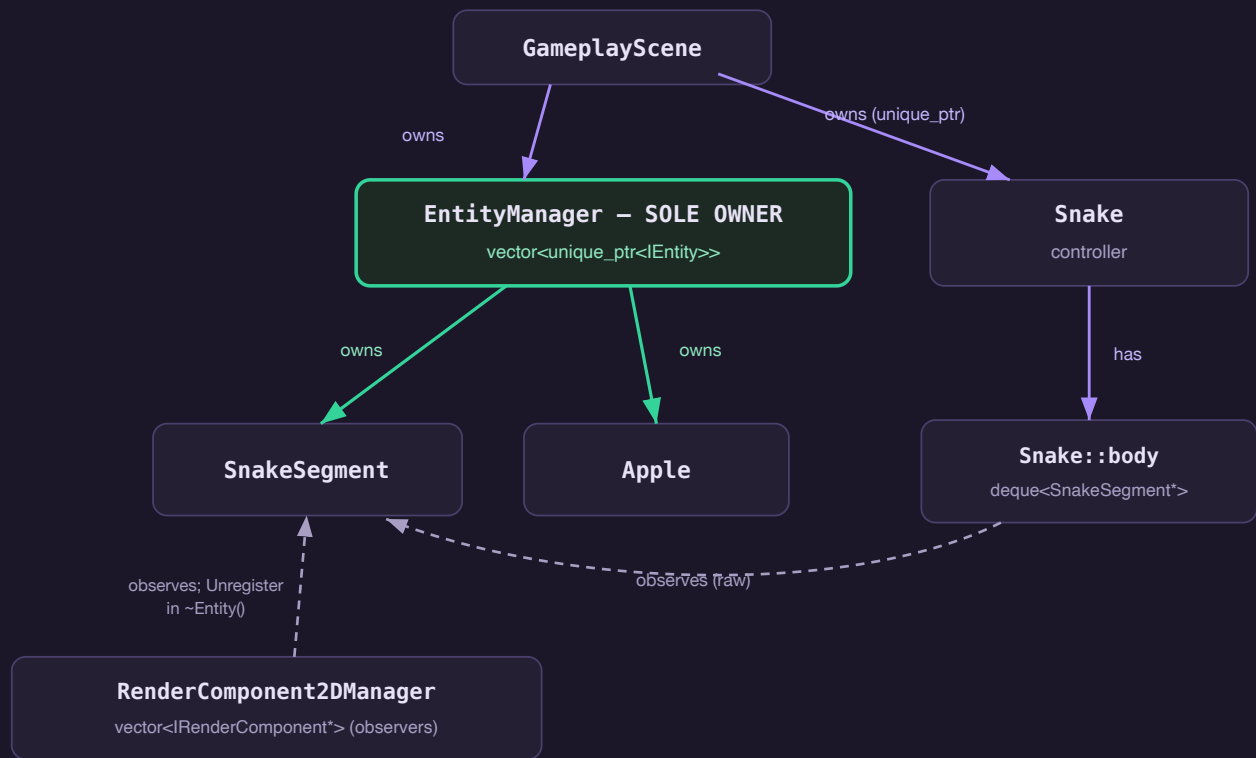
Three lifetime models point at the same objects. `SnakeSegment` has **two** owners; `Apple` and its transform have **none**; the render manager holds raw pointers into objects owned by someone else.



Reading the diagram. Solid purple = ownership (`unique_ptr` / by-value). Dashed red = a non-owning raw pointer that *acts* like a claim. `SnakeSegment` is the collision point: `Snake::body` owns it via `unique_ptr` while `EntityManager` also holds a raw `IEntity*` to it — the moment the manager gets a real destructor (the natural fix for the `Apple` leak), the segment is freed twice. `Apple` and its transform have no owner at all, so they leak today. The app-lifetime `RenderComponent2DManager` keeps raw pointers into per-entity components, so on scene teardown they dangle.

3. Target — one owner, observers everywhere else

`EntityManager` becomes the **sole owner** of every entity (segments *and* apple). `Snake` becomes a controller that *observes* its segments. Managers keep non-owning observers and are told to `Unregister` when the owner destroys the object.



Legend. ■ green = ownership transferred to the single owner · ■ purple = `unique_ptr` ownership · ■ dashed grey = non-owning observer. There is now exactly one solid arrow into every object, and observers can never outlive their target because the owner's destructor drives de-registration.

4. The shape, in code

These are **illustrative sketches** against the current types, not applied edits.

4.1 `EntityManager` becomes the sole owner

```

// entity-manager.hpp
// sink: caller hands over ownership; returns a non-owning handle
Core::Entities::IEntity* AddEntity(std::unique_ptr<Core::Entities::IEntity> entity);
void RemoveEntity(Core::Entities::IEntity* entity);
private:
    std::vector<std::unique_ptr<Core::Entities::IEntity>> entities; // was vector<IEntity*>
  
```

```

IEntity* EntityManager::AddEntity(std::unique_ptr<IEntity> e) {
    IEntity* handle = e.get();
    entities.push_back(std::move(e));
    return handle;
}
void EntityManager::RemoveEntity(IEntity* e) {
    std::erase_if(entities, [&](const auto& p){ return p.get() == e; });
    // ~Entity() runs here → its components unregister themselves (§4.4)
}
  
```

4.2 GameplayScene::OnEnter — no raw new

```
this->snake = std::make_unique<Snake>(SnakeParams{ .settings = gameSettings, /* ... */ });
this->snake->Initialize(this->entityManager);           // creates segments INTO the manager

auto apple = std::make_unique<Apple>(AppleParams{ /* position value */ });
this->entityManager.AddEntity(std::move(apple));
```

The stray `new Engine::Spatial::Transform2D(...)` disappears — `Apple` builds and owns its own `TransformComponent2D` from a **value**, which also removes F3's dangling stack-temp and that transform leak.

4.3 Snake creates into the manager, keeps observers

```
// snake.hpp
std::deque<SnakeSegment*> body;    // was deque<unique_ptr<SnakeSegment>>
SnakeSegment* head{};
```

```
// snake.cpp
void Snake::CreateBody(EntityManager& em) {
    for (int i = 0; i < length; i++) {
        Engine::Spatial::Transform2D t = /* per-segment value */;
        auto seg = std::make_unique<SnakeSegment>(SnakeSegmentParams{ i, t });
        SnakeSegment* handle = static_cast<SnakeSegment*>(em.AddEntity(std::move(seg)));
        body.push_back(handle);
    }
    head = body.front();
}
```

Growth (F4) now reads each segment's **own** transform, not a shared function-local `static` or an address into a sibling segment.

4.4 The linchpin — symmetric register / unregister

```
RenderComponent2D::~~RenderComponent2D() { manager->Unregister(this); } // observer removed bef
```

Give the component a back-pointer to its manager (or drive it from `~Entity()` if entities, not components, should own that relationship). Wherever you `Register`, guarantee the matching `Unregister` runs on destruction.

4.5 Fix F1 while you're here — Unregister takes an observer

```
void Register(std::unique_ptr<IRenderComponentUI> component); // sink (takes ownership)
void Unregister(IRenderComponentUI* component);               // observer (no ownership)
// impl: std::erase_if(renderComponents, [&](auto& up){ return up.get() == component; });
```

5. What this dissolves

Finding	How the single-owner model resolves it
● F2 — double free / leak / UAF	One owner (<code>EntityManager</code>). <code>Snake::body</code> no longer owns $\Rightarrow$ no double free; <code>Apple</code> + transform now owned $\Rightarrow$ no leak
● F1 — <code>Unregister</code> double-free	<code>Unregister</code> takes an observer, never a second <code>unique_ptr</code>
● F8 — null <code>entityManager</code> deref (live crash)	Delete <code>GameplayStateMachine::entityManager</code> ; the scene's manager is the only one — the null-deref path is gone
● F9 — two entity managers	Single manager owned by the scene; state machine no longer duplicates it
● F3 — dangling stack transform	Entities own transforms by value; no pointer to a temporary
● F4 — shared <code>static</code> transform	Per-entity transform storage; no aliasing
● F13 — segment transform built twice	Built once, owned by the entity
— Transient scene re-entry	Destroying an entity unregisters its render component, so <code>SwitchTo</code> cleans up the app-lifetime manager instead of leaving dangling pointers

6. Destruction-order caveat

`Snake`'s observers point into entities owned by `EntityManager`. Declare `entityManager` **before** `snake` in the scene (members destroy in reverse declaration order), or `snake.reset()` first, so the observer-holder tears down before the owner. Since `~Snake()` only drops raw pointers (never dereferences them), order is not fatal — but the invariant to preserve is: *observers never touch an object after its owner is gone*.

7. Suggested migration order

1. `EntityManager` ownership (§4.1) + delete raw `new` in `gameplay-scene.cpp` (§4.2).
2. `Snake` observers (§4.3) — segments created into the manager.
3. Register/Unregister discipline (§4.4) + **F1 signature** (§4.5).
4. Delete `GameplayStateMachine::entityManager` (F8/F9) — crash removed *properly*.
5. Rebuild under **AddressSanitizer** and drive a scripted play-through; ASan now reports any residual aliasing deterministically.